

Programming Paradigm

The Village app will use a hybrid programming paradigm of functional and data-oriented with potential limited object-oriented principles.

The primary paradigm will be functional and will be used for the business logic including data transformations, Express middleware chains and React functional components with hooks.

An example React component may look like the following:

```
import { useState } from 'react';

function MoodLogger() {
  const [mood, setMood] = useState('');

  const logMood = () => fetch('/api/moods', {
    method: 'POST',
    body: JSON.stringify({ mood })
  });

  return (
    <input value={mood} onChange={e setMood(e.target.value)} />
    <button onClick={logMood}>Log Mood</button>
  );
}
```

This is correct for our MVP as it has lower complexity with easier unit testing. It's faster to develop with reduced architectural overhead and minimal boilerplate. The functional paradigm keeps logic deterministic and predictable which is critical for mental health, pain and event tracking systems where calculations must be reliable.

Secondary to this, we will use data-oriented design for MongoDB schema modeling and document structuring around usage patterns. As MongoDB is document-based, it is not relational. Therefore, data is structured by access pattern (e.g.: mood logs and pain logs embedded by the user), queries drive the schema design and aggregations support analytics. For our MVP, data modeling will prioritise simplicity and retrieval efficiency.

```
// models/MoodEntry.js
const mongoose = require('mongoose');

const MoodEntrySchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  value: { type: Number, required: true },
  createdAt: { type: Date, default: Date.now }
});

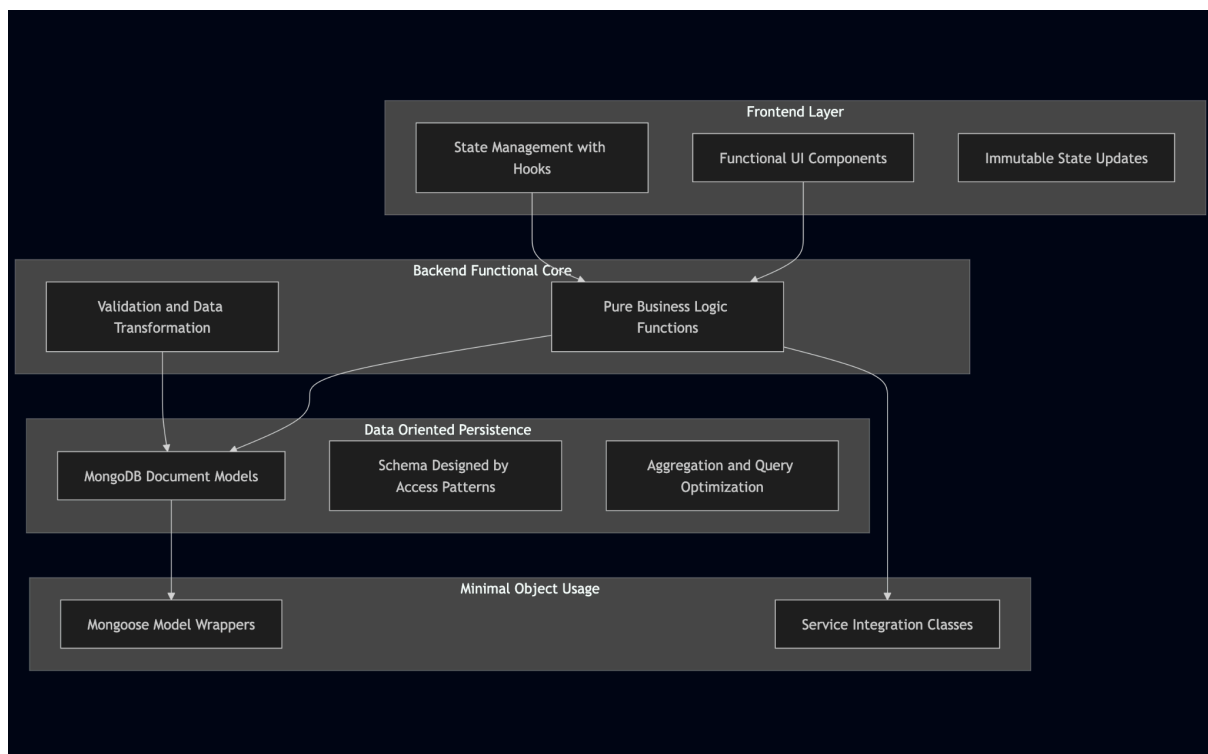
module.exports = mongoose.model('MoodEntry', MoodEntrySchema);
```

Lastly, we will have limited object-oriented usage and it's patterns will only appear in mongoose models for schema abstraction, and possibly service wrappers down the road of development. There will be no deep inheritance or domain entity class hierarchies. Our MVP will avoid heavy use of OOP as it increases complexity early on with little benefit. As we have floated the idea of email notifications, an example of this in OOP could look like this:

```
// services/EmailService.js
class EmailService {
  constructor(smtpConfig) { this.config = smtpConfig; }
  sendEmail(to, subject, body) { /* some logic here */ }
}
module.exports = EmailService;
```

In short, our system uses a functional-first approach for business logic and front end components, data-oriented principles for MongoDB schema design and minimal OOP abstractions limited to schema definitions and potential service boundaries.

An example diagram of how this paradigm works together may look like this:



Application Architecture

The Village application will opt for a monolithic model-view-controller architecture. In comparison, microservices would introduce network overhead, service discovery complexity, deployment orchestration and increased DevOps requirements. As such, an MVP does not justify this complexity.

Our system architecture model will apply Mongoose schemas (User, MoodLog, PainLog, EventLog) with database validation rules and data access methods. This layer will handle persistence only. The model layer might look like this for example:

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: false,
  }
});

const UserModel = mongoose.model("User", UserSchema);

module.exports = {
  UserSchema,
  UserModel
}
```

Next, our View will render user interface and handle interaction with a React front end, functional components, state management via hooks and API consumption.

```
<MoodLogger /> // as shown above
```

Controllers must remain thin to coordinate. Ours will handle Express routes, request validation, calls to service/ business logic and JSON response formatting. For example:

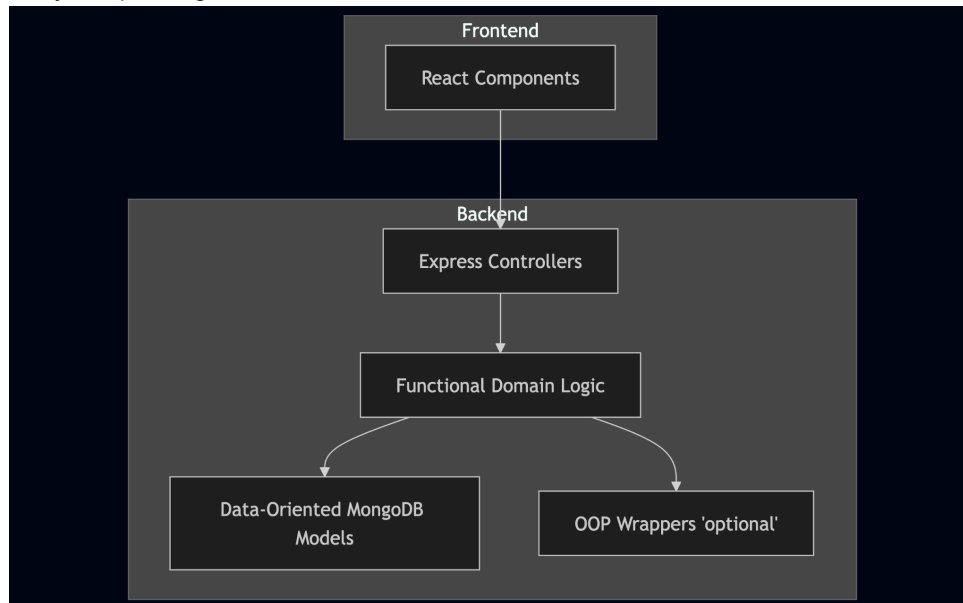
```
// controllers/pain.controller.js
const PainEntry = require('../models/PainEntry');
const { calculatePainScore } = require('../domain/pain.logic');

async function logPain(request, response) {
  const entry = await PainEntry.create(request.body);
  const score = calculatePainScore(await PainEntry.find({ userId:
request.body.userId }));
  response.json({ entry, score });
}
module.exports = { logPain };
```

MVC is the correct option for our MVP as it's core features are user authentication, mood, pain and event tracking. As such, we need clear separation of concerns, a predictable request flow, fast iteration and maintainability for scaling. MVC provides logical organisation and is simpler for onboarding team members. It is also the industry-standard pattern for MVP SaaS applications. Here is simple overview of the system:



The layered hybrid paradigm in MVC:



And an example of the full application architecture:

